

The Master HTTP Status Codes Reference Guide

The Complete Full-Stack Developer & Technical Interview Handbook

Categorized by Status Code Families with Real-World Practical Examples

The 5 Status Code Families at a Glance

Family	Category	Plain English Meaning
1xx	Informational	"Hold on, I'm processing your request..."
2xx	Success	"Success! Everything worked as expected."
3xx	Redirection	"Go over there, the resource has moved."
4xx	Client Error	"You (the browser/user) made a mistake."
5xx	Server Error	"We (the backend/database) crashed or broke."

1. The 2xx Family: SUCCESS

Used when the server successfully received, understood, and accepted the request.

Status Code	Name	Practical Reason & When to Use	Real-World REST Example
200	OK	Standard successful response for reading or modifying data.	GET /api/persons returns array of contacts. PUT /api/persons/:id updates a contact.
201	Created	A brand-new resource was successfully created and persisted on the server.	POST /api/persons creates a new person and returns the new object with its generated ID.
204	No Content	The request succeeded, but there is intentionally zero body data to return.	DELETE /api/persons/:id deletes a record. (Server sends empty body).

2. The 3xx Family: REDIRECTION & CACHING

Used when the client must take additional action to complete the request.

Status Code	Name	Practical Reason & When to Use	Real-World REST Example
301	Moved Permanently	The requested URL has permanently moved to a new	Accessing http://mywebsite.com automatically redirects permanently to https://mywebsite.com .

Status Code	Name	Practical Reason & When to Use	Real-World REST Example
		address. Browsers cache this redirect.	
302	Found (Temporary)	The resource is temporarily located at a different URL.	Redirecting an unauthenticated user temporarily to <code>/login</code> .
304	Not Modified	The cached version in the browser is still fresh and valid. Saves network bandwidth.	Browser requests an image; server responds 304 so browser uses local disk cache.

3. The 4xx Family: CLIENT ERRORS (The User's Mistake)

Used when the client sent invalid input, failed authentication, or requested missing resources.

Status Code	Name	Practical Reason & When to Use	Real-World REST Example
400	Bad Request	The client provided invalid syntax, malformed JSON, or missing required fields.	<code>POST /api/persons</code> sent without a <code>name</code> or <code>number</code> property.
401	Unauthorized	The user is NOT authenticated (missing, expired, or invalid login token).	Trying to fetch private account settings without a valid JWT token.
403	Forbidden	The user IS authenticated, but does NOT have permission/privileges for this action.	A normal customer trying to delete another user's account or view <code>/admin</code> .
404	Not Found	The target endpoint URL or specific resource ID does not exist on the server.	<code>GET /api/persons/99999</code> where ID 99999 does not exist in the database.
409	Conflict	The request conflicts with current server state (commonly duplicate unique constraints).	Registering a new account with an email address that is already registered.
422	Unprocessable Entity	Syntax is valid JSON, but the data fails business validation rules.	Submitting a password that is only 2 characters long when 8 are required.

4. The 5xx Family: SERVER ERRORS (The Backend's Mistake)

Used when the backend server encountered an unexpected failure, crash, or timeout.

Status Code	Name	Practical Reason & When to Use	Real-World REST Example
500	Internal Server Error	An unhandled exception or bug crashed your Node.js JavaScript execution.	Calling <code>.map()</code> on <code>undefined</code> inside <code>index.js</code> , or database connection failed.
502	Bad Gateway	A proxy server (NGINX, Cloudflare, Render) received an invalid response from your Node app.	Cloudflare is online, but your Express container crashed on startup.
503	Service Unavailable	The server is temporarily down due to maintenance or massive traffic overload.	Server CPU hits 100% capacity during a Black Friday flash sale spike.
504	Gateway Timeout	The backend server took too long to complete the request and timed out.	A slow database query takes longer than 30 seconds to complete.

💡 The 3 Most Commonly Confused Pairs

1. 401 Unauthorized vs 403 Forbidden

- **401 Unauthorized:** "I don't know who you are. Please log in first." (Identity Unknown).
- **403 Forbidden:** "I know who you are, but you are not allowed to touch this." (Identity Known, Permission Denied).

2. 200 OK vs 201 Created vs 204 No Content

- **200 OK:** "Request succeeded; here is your payload." (Used for **GET** and **PUT**).
- **201 Created:** "Resource successfully created; here is the new item with its ID." (Used for **POST**).
- **204 No Content:** "Resource successfully deleted; there is nothing to return." (Used for **DELETE**).

3. 400 Bad Request vs 404 Not Found

- **400 Bad Request:** "You sent bad data." (Missing required form fields or bad JSON).
- **404 Not Found:** "The route or resource ID doesn't exist." (Typo in URL or wrong ID).

📄 Quick-Lookup Cheat Sheet